Authentication

System Description

Abstract

This document describes the Authentication core system of the Eclipse Arrowhead framework (documentation edition 5.3.0), as implemented in Java Spring in `ah5-core-java-spring/authentication`, including the local extensions of this workspace.

Contents

1 Documentation edition and implementation scope **3**

2 Overview **3**
2.1 Significant Prior Art 4
2.2 How This System Is Meant to Be Used 4
2.3 System functionalities and properties 4
2.4 Important Delimitations 5

3 Services **6**
3.1 Produced services 6
3.2 Consumed services 6

4 Security **7**
4.1 Security model 7

5 References **7**

6 Revision History **9**
6.1 Amendments 9
6.2 Quality Assurance 9

1 Documentation edition and implementation scope

This is the 5.3.0 documentation edition, dated 2026-11-09. It carries forward the system and service contracts from the retained 5.2.2 documentation and identifies the current primary implementation sources. The former `documentation/v5.2.2` folder is retained unchanged.

Document version 5.3.0 is distinct from executable, dependency, registered system/service and wire-contract versions. Values still defined by the source (including 5.2.2 artefact versions, 1.0.0 service versions and `SysMonitor-AMQP-IDD-5.2.2` payload identifiers) are preserved. This documentation update does not change runtime behaviour or establish a new implementation release. Historical verification statements describe the tests recorded in the inherited baseline; this revision validates document compilation and references.

2 Overview

This document describes the Authentication system. Authentication is the identity provider of an Arrowhead 5 local cloud. It stores the identities of the systems with their credentials and sysop flag, lets systems log in and out, and issues and verifies the identity tokens with which systems authenticate towards each other.



Figure 1: SysML-style block view of Authentication with its produced and consumed services.

The rest of this document is organized as follows. In Section 2.1, we reference major prior art capabilities of the system. In Section 2.2, we describe the intended usage of the system. In Section 2.3, we describe fundamental properties provided by the system. In Section 2.4, we describe delimitations of capabilities of the system. In Section 3, we describe the abstract service functions consumed or produced by the system. In Section 4, we describe the security capabilities of the system.

2.1 Significant Prior Art

Arrowhead 4.x relied on X.509 client certificates alone. Arrowhead 5 introduces Authentication with pluggable authentication methods (password is implemented) and identity tokens; certificate-based authentication remains available to the other systems.

2.2 How This System Is Meant to Be Used

An operator (or the onboarding procedure) creates the identity of a system with `identityManagement`. The system logs in with `identity.identity-login`, sends the token in the `Authorization` header of its requests, and renews it before expiry. Providers verify the token with `identity-verify` or, if they hold the Authentication secret key, by checking the `AUTHENTICATOR-KEY` locally.

2.3 System functionalities and properties

2.3.1 Functional properties of the system

- Password authentication of systems (method `PASSWORD`), with hashed password storage.
- Issue, verify and revoke identity tokens; one active session per system.
- Change of credentials by the system itself.
- Management of identities and sessions; sysop flag for system operators.
- Periodic removal of expired sessions (`cleaner.job.interval`).
- Binding of login, logout and credential change, and of every token use, to the client certificate of the same system (local extension).

2.3.2 Configuration of system properties

The system is configured with Spring properties (`application.properties`, overridden by environment variables in the container deployment). The properties that shape its behaviour are:

Property	Default	Meaning
`server.port`	8444	HTTPS port.
`authentication.policy`	internal	Must stay `internal`: Authentication checks tokens in its own database.
`arrowhead.security.profile`	secure	`development` selects the isolated development exception; never set in a deployment.
`authentication.secret.key`	(per-cloud secret)	Key with which Authentication signs its own requests (`AUTHENTICATOR-KEY`).
`identity.token.duration`	300	Token lifetime in seconds (built-in default 600).
`cleaner.job.interval`	60000	Session cleaner interval in ms.
`management.policy` / `management.whitelist`	authorization	Access to `identityManagement` and `generalManagement`.
`http.collector.bootstrap.services`	..., authorization	Services looked up directly in the ServiceRegistry (local).
`normalization.mode`	extended	Name normalization.
`mqtt.api.enabled`	false	Offer the MQTT interfaces.
`sysmonitor.enabled`	false	Publish native sysMonitor observations (test cloud: true).
`arrowhead.ssl.crl-file`	(unset)	CRL checked for every client certificate (local).

Secrets (database, keystore and identity passwords, secret keys) have no usable defaults; in the test clouds they are per-cloud secrets mounted as files.

2.3.3 Data stored by the system

Identities (system name, sysop flag, authentication method, creator and update data), password hashes, active sessions with token and expiry, and the log entries (database `ah_authentication`).

2.3.4 Non functional properties

Every authenticated request in the cloud depends on a valid token; with `outsourced` providers each token is verified once per request unless checked with the secret key. Tokens are short-lived (300 s), which limits the effect of a stolen token.

2.3.5 Stateful or stateless

Stateful: identities and sessions are persistent; a restart keeps the sessions valid until they expire.

2.4 Important Delimitations

Authentication does not authorize: the sysop flag is the only privilege it records. It does not issue certificates. Only the password method is implemented. Because one session is kept per system, a second login of the same system invalidates the token of the first.

3 Services

This section describes the produced and consumed services. Each produced service is specified by an SD document and one IDD document per interface.

3.1 Produced services

Service	Version	Interfaces	SD	IDD
`identity`	1.0.0	HTTP(S), MQTT(S)	[1]	[2], [3]
`identityManagement`	1.0.0	HTTP(S), MQTT(S)	[4]	[5], [6]
`generalManagement`	1.0.0	HTTP(S), MQTT(S)	[7]	[8], [9]
`sysMonitor`	1.0.0	AMQP(S)	[10]	[11]

The HTTP interfaces are always offered, the MQTT interfaces with `mqtt.api.enabled=true` and the AMQP `sysMonitor` interface with `sysmonitor.enabled=true`. Upstream 5.2.2 also answers `GET <base>/monitor/echo` on some systems; this endpoint is not registered as a service.

3.2 Consumed services

Provider	Service	Operations	Use	Condition
ServiceRegistry	serviceDiscovery	register, revoke, lookup	Registration of its services; lookup of core services (bootstrap services).	always
Consumer Authorization	authorization	verify	Management filter decisions (looked up in the ServiceRegistry, local configuration).	`management.policy=authorization`
AMQP broker (SysMonitor)	sysMonitor (publisher)	echo, power, status	Own observations.	`sysmonitor.enabled=true`

Except for the bootstrap services (`http.collector.bootstrap.services`), which are looked up in the ServiceRegistry, the system reaches other systems' services through DynamicServiceOrchestration (`http.collector.mode=ORCHESTRATION`, results cached for `http.collector.orchestration.ttl` = 300 s); every such call needs a ConsumerAuthorization grant.

4 Security

The default is the Arrowhead secure mode with HTTPS and X.509 certificates following the Arrowhead certificate profile (`<System>.<Cloud>.<Organization>.arrowhead.eu`). Certificates not issued under the cloud CA are refused by the TLS handshake. The Arrowhead-test-cloud-java deployment runs the secure mode.

The packaged `application.properties` is a development sample that disables TLS and declares identities. The system refuses to start with it, or with any other effective configuration that sets `server.ssl.enabled=false`, `authentication.policy=declared`, `service.authorization.enabled=false` or, while monitoring is on, `sysmonitor.ssl.enabled=false`. The check runs before the system accepts a connection. Only an explicit `arrowhead.security.profile=development` lets such a system start, for an isolated test, and it then logs a non-conformance warning.

4.1 Security model

- **Protocols:** HTTPS (TLS 1.3 and 1.2) with mandatory client certificates (`server.ssl.client-auth=need`); MQTT over TLS; AMQP over TLS for `sysMonitor`.
- **Data protection:** transport encryption by TLS; the database connection uses TLS with server verification in the test cloud. Secrets are kept out of the container environment (Compose file secrets).
- **System authentication:** by the authentication policy: `certificate` (client certificate CN), `outsourced` (identity token verified with Authentication), `internal` (Authentication only) or `declared` (insecure). In this workspace an identity token is accepted only together with the client certificate of the system it was issued to (token-to-certificate binding), and every client certificate is checked against the cloud CA's CRL when `arrowhead.ssl.crl-file` is set (fail closed).
- **Authorization of produced services:** management services are protected by the management filter (`management.policy`: `SYSOP_ONLY`, `WHITELIST` or `AUTHORIZATION` via ConsumerAuthorization); an optional blacklist filter refuses blacklisted requesters. Sysop identities bypass the management filter.
- Authentication uses the `internal` authentication policy: a requester's token is looked up in its own session table. Over HTTPS a system can only log in, log out or change credentials as the system named by its client certificate, and every token must be presented with the certificate of the system it belongs to. Passwords are stored only as hashes.

For the Arrowhead certificate profile see github.com/eclipse-arrowhead/documentation.

5 References

[1] J. Delsing, "identity – Service Description (Authentication)," Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-identity-SD.pdf.

[2] ——, "identity HTTP/TLS/JSON – Interface Design Description (Authentication)," Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-identity-HTTP-IDD.pdf.

[3] ——, "identity MQTT/TLS/JSON – Interface Design Description (Authentication)," Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-identity-MQTT-IDD.pdf.

[4] ——, "identityManagement – Service Description (Authentication)," Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-identityManagement-SD.pdf.

[5] ——, "identityManagement HTTP/TLS/JSON – Interface Design Description (Authentication)," Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-identityManagement-HTTP-IDD.pdf.

[6] ——, "identityManagement MQTT/TLS/JSON – Interface Design Description (Authentication)," Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-identityManagement-MQTT-IDD.pdf.

[7] ——, “generalManagement – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-generalManagement-SD.pdf.

[8] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-generalManagement-HTTP-IDD.pdf.

[9] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-generalManagement-MQTT-IDD.pdf.

[10] ——, “sysMonitor – Service Description (Authentication),” Eclipse Arrowhead, SD 5.3.0, 2026, authentication-v5.3.0-sysMonitor-SD.pdf.

[11] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (Authentication),” Eclipse Arrowhead, IDD 5.3.0, 2026, authentication-v5.3.0-sysMonitor-AMQP-IDD.pdf.

6 Revision History

6.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-11-07	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-11-09	5.3.0	New documentation edition; primary-source references and Go/Rust implementation scope; 5.2.2 retained.	Documentation maintenance

6.2 Quality Assurance

No.	Date	Version	Approved by
1		5.3.0	